



Lista de Exercícios – Vetor e Matriz

1 - Escreva um programa em Python que solicite ao usuário a quantidade N de números inteiros que deseja cadastrar. Construa uma lista vazia e preencha-a dinamicamente em tempo de execução utilizando o método `append()`. Em seguida, percorra a lista e exiba: (a) o maior e o menor valor digitados; (b) a média aritmética dos elementos; e (c) a quantidade de números pares

2 - Dada a lista estática de salários `salarios = [2500.0, 3200.0, 1800.0, 4500.0, 2100.0]`: (a) Atualize o salário no índice 2 para 2300.0; (b) Peça ao usuário um valor de salário para buscar e verificar se existe na lista usando o operador `'in'`; (c) Se existir, remova-o usando `.remove()`; (d) Extraia e remova o último elemento da lista utilizando `.pop()` e exiba o elemento removido.

3 - Crie um vetor com 10 números inteiros de 1 a 10. Utilizando a técnica de fatiamento (slicing) do Python: (a) Gere uma sublista contendo apenas os elementos localizados nos índices pares; (b) Gere uma sublista contendo apenas os números ímpares (por valor); (c) Crie uma cópia da lista inteira invertida sem utilizar o método `.reverse()`.

4 - Utilizando a biblioteca NumPy: (a) Crie um array 1D com os números inteiros de 10 a 50 (inclusive) com passo 5 usando `np.arange()`; (b) Eleve todos os elementos do array ao quadrado de forma vetorizada (sem laço `'for'`); (c) Aplique uma filtragem por indexação booleana para extrair apenas os valores superiores a 500.

5 - Considere o array NumPy `notas = np.array([5.5, 8.0, 4.5, 9.0, 6.5])`: (a) Insira a nota `'7.0'` no índice 2 utilizando `np.insert()`; (b) Substitua todas as notas inferiores a `'6.0'` pela nota mínima de recuperação `'6.0'` usando indexação booleana; (c) Remova a nota do índice 4 utilizando `np.delete()`.

6 - Construa um programa em Python nativo para ler uma matriz 3x3 de inteiros digitados pelo usuário: (a) Preencha a matriz utilizando laços aninhados em listas de listas (`'list[list]'`); (b) Imprima a matriz em formato tabular formatado; (c) Exiba os elementos da diagonal principal (onde índice da linha $i ==$ coluna j) e da diagonal secundária (onde $i + j == 2$).

7 - Dada a matriz nativa 3x2 `matriz = [[1, 2], [3, 4], [5, 6]]`: (a) Permita que o usuário atualize o valor da linha 1, coluna 0 para 99; (b) Escreva uma função que construa e retorne a matriz transposta (tamanho 2x3), onde as linhas da matriz original viram colunas.

8 - Utilizando o NumPy: (a) Crie uma matriz 4x4 preenchida com números inteiros aleatórios entre 10 e 99 (`np.random.randint()`); (b) Calcule a soma de cada linha

(axis=1) e a média de cada coluna (axis=0); (c) Altere a forma da matriz (reshape) de 4x4 para 2x8 e exiba a nova estrutura.

9 - Desenvolva um programa completo em Python nativo para controle de estoque de uma empresa. Utilize duas listas paralelas: `produtos` (strings) e `quantidades` (inteiros). Implemente um menu interativo dentro de um laço `while` com as opções: 1. Cadastrar Produto; 2. Listar Estoque; 3. Atualizar Quantidade; 4. Excluir Produto; 5. Sair. Garantir a integridade dos índices sincronizados entre as duas listas.

10 - Um sistema acadêmico armazena as notas de 4 alunos em 3 avaliações através de uma matriz NumPy 4x3: (a) Dado a matriz de notas iniciais `notas = np.array([[5.0, 7.5, 6.0], [8.0, 9.0, 4.0], [3.5, 5.0, 6.5], [7.0, 6.0, 8.5]])`, calcule e exiba a média final de cada aluno; (b) Identifique qual avaliação (coluna) obteve a maior média geral da turma usando `np.argmax()`; (c) Aplique um reajuste automático atribuindo a nota mínima `6.0` a todas as avaliações com nota inferior a 6.0; (d) Remova a última coluna (Avaliação 3) da matriz utilizando `np.delete(..., axis=1)`.